Universidad Rafael Landívar.
Facultad de Ingeniería.
Ingeniería en Informática y Sistemas.
Laboratorio de Estructura de Datos I, Sección 01.
Docente: Ing. René Daniel Mejía Alvarado.

Proyecto de Aplicación No. 1

"Motor Inteligente de Gestión de Tráfico Urbano"

Arrazola Escobar, Luna Samantha 1022725
Donadio Castellanos, Marco Fabrizzio 1076925

Guatemala, 08 de abril de 2026.

Tabla de contenido

1. Introducción
2. Planteamiento del problema, objetivos y alcance
3. Arquitectura general del sistema
4. Programación genérica, delegación e inyección de comportamiento
5. Modelado del dominio del problema
6. Explicación detallada de las estructuras implementadas
7. Simulación y benchmarking
8. Análisis formal de complejidad
9. Buenas prácticas y decisiones de ingeniería
10. Manual de funcionalidad
11. Conclusiones generales

1. Introducción

El proyecto Motor Inteligente de Gestión de Tráfico Urbano (MIGTU) se pensó como una aplicación académica orientada a la integración de estructuras de datos avanzadas dentro de un contexto de simulación realista. La problemática presentada fue la gestión urbana del tránsito, debido a que combina de forma natural jerarquías, búsquedas, priorización dinámica de eventos y necesidad de analizar desempeño bajo alta carga. En términos computacionales, el dominio del tráfico urbano resulta especialmente apropiado para demostrar por qué la selección de estructuras de datos influye directamente en la eficiencia, la mantenibilidad del código y la calidad de las decisiones operativas.

Desde la perspectiva del curso, el valor principal del proyecto no radicó únicamente en que el sistema compilara o que mostrara resultados correctos, sino en que la solución permitiera justificar formalmente las decisiones de diseño. Por esta razón, se desarrollaron desde cero un Árbol Binario de Búsqueda, un Árbol AVL, un Árbol Multi-camino y una Cola de Prioridad basada en Heap, además de clases de apoyo para modelar intersecciones, sensores, eventos, comparadores, generación de datos y mediciones de tiempo. La implementación se orientó a cumplir el requerimiento de aplicar programación genérica, uso de Comparable y Comparator, expresiones lambda, métricas internas y benchmarking automatizado.

En el desarrollo del sistema se buscaron alcanzar dos objetivos paralelos. El primero fue funcional: representar una ciudad con distritos, zonas, avenidas e intersecciones; cargar datos; procesar eventos críticos y visualizar el comportamiento del sistema. El segundo fue analítico: estudiar el costo temporal y espacial de cada estructura, contrastar el caso promedio y el peor caso, y observar experimentalmente cómo la teoría se refleja en las pruebas de inserción, búsqueda, balanceo y procesamiento de eventos. De esta manera, el proyecto sirvió tanto como implementación de software como ejercicio formal de análisis algorítmico.

El presente documento técnico expone de manera integral lo que se realizó, cómo se implementó y por qué se tomaron determinadas decisiones. Se desarrollan fundamentos teóricos, descripción de clases, análisis de complejidad, explicación del benchmarking, buenas prácticas de programación y un manual de funcionalidad detallado.

2. Planteamiento del problema, objetivos y alcance

2.1 Problema a resolver

La municipalidad de una ciudad requiere un sistema capaz de organizar la red vial, administrar intersecciones y sensores, procesar eventos de tráfico con distintos niveles de urgencia y responder eficientemente aun cuando el volumen de datos aumenta de manera considerable. El problema no es solamente almacenar información, sino hacerlo con estructuras cuya lógica interna se adapte a la naturaleza de las operaciones más frecuentes.

En un escenario urbano, la información no es homogénea. Las intersecciones se consultan por identificador, la ciudad se interpreta como una jerarquía territorial y los eventos deben atenderse según prioridad y no según el orden en que llegan. Esto obliga a seleccionar estructuras distintas para necesidades distintas: árboles binarios para indexación, árboles balanceados para estabilidad bajo carga, árboles N-arios para representar jerarquías y heaps para decidir qué evento atender primero.

2.2 Objetivo general

Desarrollar un sistema inteligente de gestión de tráfico urbano en Java que integre estructuras de datos avanzadas implementadas desde cero, permita simular el comportamiento de una red vial y haga posible el análisis formal y experimental de su rendimiento.

2.3 Objetivos específicos

- Implementar un BST genérico para indexar intersecciones y evaluar su comportamiento en caso promedio y peor caso.
- Implementar un AVL genérico que mantenga balance automático y permita comparar su altura y desempeño frente al BST.
- Diseñar un árbol multi-camino que modele de forma explícita la jerarquía urbana: ciudad, distritos, zonas, avenidas e intersecciones.
- Implementar una cola de prioridad basada en heap para gestionar eventos críticos de tráfico con prioridad dinámica.
- Aplicar programación genérica, lambdas, comparadores e inyección de comportamiento para hacer flexible el ordenamiento.
- Construir un módulo de benchmarking que mida tiempos con System.nanoTime(), exporte resultados y permita interpretar la teoría con evidencia práctica.
- Integrar todas las piezas en una aplicación de consola que pueda demostrarse durante la defensa y que sea entendible para un usuario académico.

2.4 Alcance del proyecto

El alcance del proyecto comprende la implementación funcional de las estructuras solicitadas, su integración en una aplicación de consola y la instrumentación necesaria para medir y exportar resultados. Se incluyeron clases auxiliares para mejorar la modularidad del diseño, como una fábrica de comparadores, un generador de datos pseudoaleatorios reproducibles, un cronómetro para benchmarking y entidades de dominio tales como Interseccion, Evento y Sensor.

No se incluyó conexión a bases de datos, interfaz gráfica, comunicación con dispositivos físicos ni lectura de sensores reales. La simulación se mantuvo intencionalmente en el ámbito académico, con datos generados por software y con énfasis en la corrección algorítmica y el análisis de complejidad.

3. Arquitectura general del sistema

La arquitectura del proyecto se diseñó con una separación clara entre estructuras de datos, entidades del dominio, utilidades de apoyo, benchmarking e interfaz de usuario. Esta división favoreció la reutilización del código y permitió que cada clase tuviera una responsabilidad concreta. En lugar de concentrar toda la lógica en una clase monolítica, se distribuyeron las tareas de manera que la construcción de datos, el ordenamiento, el balanceo, la medición y la presentación de resultados pudieran evolucionar por separado.

En el núcleo del sistema se encuentran las estructuras de datos: BST, AVL, ArbolN_ario y ColaPrioridad. Sobre estas estructuras se apoyan las entidades del dominio, principalmente Interseccion, Sensor y Evento. Encima de ambas capas se encuentran clases utilitarias como CentroComparaciones, GeneradorDatos y Cronometro. Luego, la clase Main funciona como orquestador del sistema, exponiendo menús de demostración, integración y benchmarking.

Este tipo de arquitectura es apropiada para un proyecto de estructuras de datos porque evita mezclar la teoría con la interfaz. Por ejemplo, un árbol no necesita conocer la manera en que el usuario selecciona una opción del menú; de igual forma, el menú no necesita conocer los detalles internos de una rotación AVL. La ventaja práctica es que la defensa del proyecto puede explicarse por componentes, mostrando cómo cada parte contribuye al comportamiento global del sistema.

Componente	Responsabilidad principal
BST / AVL	Indexación de intersecciones, búsqueda, inserción, eliminación, altura, comparaciones y balanceo.
Árbol N-ario	Representación jerárquica de la ciudad y consultas estructurales por niveles.
ColaPrioridad	Gestión de eventos críticos con extracción según prioridad dinámica.
Interseccion / Sensor / Evento	Modelado del dominio urbano y de los datos operativos del sistema.
CentroComparaciones	Fábrica de comparadores reutilizables e inyección de comportamiento.
GeneradorDatos	Generación controlada de datos para simulación, demos y benchmarking.
Cronometro / BenchMark	Medición experimental, cálculo de ns/op, impresión y exportación a CSV.
Main	Interfaz por consola, demostraciones, integración y flujo general del sistema.

4. Programación genérica, delegación e inyección de comportamiento

Uno de los requerimientos formales del proyecto fue el uso de programación genérica y delegación. Esta exigencia no se cumplió únicamente como un requisito sintáctico, sino como una decisión de diseño. Las estructuras principales se implementaron con parámetros genéricos <T>, lo que permitió que un mismo árbol o heap pudiera trabajar con distintos tipos de objetos sin quedar amarrado exclusivamente a Interseccion o Evento.

En teoría, la programación genérica permite escribir algoritmos independientes del tipo concreto de dato. En la práctica, esto reduce duplicación, mejora reutilización y obliga a diseñar interfaces más limpias. El BST y el AVL, por ejemplo, no necesitan conocer detalles internos de una intersección; solo necesitan un mecanismo para comparar dos elementos. Ese mecanismo se logró mediante Comparator<T> y, en algunos casos, mediante Comparable<T> como orden natural de la entidad.

La delegación de comportamiento se aplicó al permitir que la lógica de comparación no estuviera escrita dentro de las estructuras. En lugar de codificar una única forma de ordenar, se inyectó el criterio desde afuera. Así, el mismo BST puede ordenar por ID, por nivel de congestión o por riesgo, según el comparador suministrado. La cola de prioridad, por su parte, puede actuar como max-heap o como min-heap dependiendo del comparador vigente.

Esta estrategia se complementó con el uso de expresiones lambda y referencias a métodos. Las lambdas permitieron crear comparadores concisos y expresivos, mientras que la clase CentroComparaciones concentró criterios reutilizables. El resultado fue un código más flexible y coherente con el propósito académico del proyecto: demostrar que una estructura bien diseñada separa la lógica algorítmica de la política de ordenamiento.

5. Modelado del dominio del problema

5.1 Intersección como entidad principal

La clase Interseccion constituye la entidad central a indexar dentro del sistema. Cada objeto representa un punto relevante de la red vial y almacena atributos de identificación, localización y estado operativo. Desde el punto de vista del diseño, se optó por que la intersección implementara Comparable<Interseccion> con orden natural por ID, debido a que dicho identificador es el criterio más estable para búsquedas directas y para la indexación primaria en árboles binarios.

La clase no se limitó a guardar datos estáticos. Además del ID, nombre, avenida, zona y distrito, incluye nivel de congestión, nivel de riesgo, estado de activación y un sensor asociado. Esta decisión permitió que la entidad no fuera simplemente una ficha descriptiva, sino un objeto con comportamiento y atributos que evolucionan dentro de la simulación. Por ejemplo, el nivel de congestión puede recalcularse dinámicamente a partir de la información del sensor.

Se redefinieron equals y hashCode con base en el ID. Esta práctica es importante porque dos objetos diferentes en memoria pueden representar la misma intersección si comparten identificador. Gracias a ello, las comparaciones lógicas dentro del sistema son consistentes y más cercanas al significado real del dato.

5.2 Sensor como fuente de comportamiento dinámico

La clase Sensor se asoció a cada intersección para modelar información operativa del tráfico. Sus atributos principales son velocidad promedio, volumen vehicular y estado del sensor. En lugar de asignar el nivel de congestión arbitrariamente, se optó por calcularlo a partir de estos datos. El enfoque seguido considera que una velocidad promedio baja y un volumen vehicular alto son indicios de mayor congestión, lo cual resulta coherente con la interpretación física del problema.

El método calcularNivelCongestion recibe una velocidad máxima y una capacidad máxima de referencia. A partir de ellas se calculan dos factores: uno derivado de la reducción de velocidad y otro derivado de la ocupación vehicular. El promedio de ambos factores se escala a un rango de 0 a 100. Esto no pretende ser un modelo de ingeniería de tráfico exacto, sino una aproximación académica razonable y suficiente para generar datos consistentes dentro de la simulación.

5.3 Evento como unidad de prioridad

La clase Evento representa incidentes o reportes que deben ser atendidos por el sistema. Se definió un enum TipoEvento con categorías como accidente grave, ambulancia en ruta, congestión alta, semáforo descompuesto o evento climático. Cada tipo posee una prioridad base y una descripción. Esta decisión de diseño fue pertinente porque centraliza la semántica del evento y evita depender de cadenas de texto ambiguas.

Aparte del tipo, cada evento almacena un identificador, una prioridad actual, el ID de la intersección asociada, una marca de tiempo y una descripción. La prioridad inicial se toma de la prioridad base del tipo, pero puede modificarse dinámicamente, lo que vuelve útil a la cola de prioridad cuando un evento cambia de severidad durante la ejecución. La clase implementa Comparable<Evento> para definir un orden natural, aunque en la práctica el proyecto privilegia comparadores externos para demostrar cambio de criterio dinámico.

6. Explicación detallada de las estructuras implementadas

6.1 Árbol Binario de Búsqueda (BST)

Un árbol binario de búsqueda es una estructura jerárquica en la cual cada nodo posee como máximo dos hijos y se cumple la propiedad de orden: todos los elementos del subárbol izquierdo son menores que el nodo actual y todos los elementos del subárbol derecho son mayores. Esta propiedad hace posible descartar una parte del árbol en cada comparación, razón por la cual el BST es una alternativa natural para indexar intersecciones por ID.

En la implementación desarrollada, la clase BST<T> almacena una referencia a la raíz, un Comparator<T> activo y varias métricas internas: inserciones totales, búsquedas totales, comparaciones totales y comparaciones de la última operación. Asimismo, se conserva una lista con el orden de inserción, útil para diagnóstico y para reinsertar datos cuando cambia el comparador. Este detalle es valioso porque muestra que una estructura no solo debe operar bien, sino también permitir su análisis experimental.

El método de búsqueda trabaja comparando el elemento objetivo con el dato del nodo actual. Si son iguales, la operación termina. Si el valor buscado es menor, la búsqueda se desplaza al subárbol izquierdo; si es mayor, al derecho. Esta lógica refleja la teoría clásica del BST y permite obtener tiempos eficientes cuando la altura del árbol se mantiene acotada. La clase contabiliza comparaciones, lo cual es pertinente para contrastar teoría y práctica durante el benchmarking.

La inserción se implementó de forma iterativa. Aunque una versión recursiva era más directa desde el punto de vista teórico, durante las pruebas se consideró más robusto evitar profundidad excesiva de pila cuando el árbol se aproxima al peor caso. El algoritmo desciende hasta encontrar la posición nula donde debe colocarse el nuevo nodo y evita insertar duplicados. Este diseño evidencia una decisión de ingeniería concreta: en contextos de carga masiva, una implementación iterativa puede ser más segura que una recursiva sin sacrificar claridad.

La eliminación contempla los tres casos clásicos. Si el nodo a eliminar no tiene hijos, simplemente se remueve. Si tiene un solo hijo, se reemplaza por dicho hijo. Si tiene dos hijos, se reemplaza por un sucesor o predecesor inorden y luego se elimina el nodo de apoyo. La versión implementada utiliza el mínimo del subárbol derecho en el BST, preservando la propiedad de orden. Esta operación es importante porque permite defender que el árbol no solo inserta y busca, sino que mantiene consistencia estructural ante actualizaciones.

Se incluyeron recorridos in-order, pre-order, post-order y por niveles. El recorrido in-order es especialmente significativo porque, si el árbol respeta la propiedad BST, devuelve los elementos en orden según el comparador activo. Los recorridos, aunque no siempre son la operación más costosa ni la más usada en producción, resultan indispensables para demostrar que la estructura interna se construyó correctamente.

De último, se implementaron métodos para calcular altura, contar nodos, contar hojas, verificar balance y mostrar estadísticas. El BST no se balancea automáticamente; sin embargo, verificar si se encuentra balanceado proporciona un indicador útil del deterioro estructural causado por ciertos patrones de inserción.

6.2 Árbol AVL

El árbol AVL es un árbol binario de búsqueda auto-balanceado. Conserva la propiedad de orden del BST, pero además impone que la diferencia entre la altura del subárbol izquierdo y derecho en cada nodo no supere una unidad. El objetivo de esta restricción es impedir que el árbol se degrade hacia una forma lineal, manteniendo con ello una altura logarítmica y, por tanto, costos de búsqueda e inserción más estables.

La implementación del proyecto hereda de BST<T>, lo cual permite reutilizar infraestructura común como la raíz, el comparador y varias métricas. No obstante, el AVL introduce responsabilidades adicionales: almacenar la altura actualizada en cada nodo, calcular el factor de balance y ejecutar rotaciones cuando detecta desbalance. Esta reutilización mediante herencia es adecuada en el contexto del proyecto porque el AVL conceptualmente sigue siendo un BST, pero con reglas suplementarias.

La inserción AVL se lleva a cabo en dos fases lógicas. La primera inserta el elemento respetando la propiedad de orden. La segunda recalcula alturas al regresar por la recursión y verifica el factor de balance de cada nodo. Si el balance queda fuera del rango permitido, se ejecuta una rotación simple o doble, según el patrón de desbalance detectado. De esta forma, la estructura corrige localmente el problema antes de que el árbol completo se deteriore.

Las rotaciones simples resuelven los casos izquierda-izquierda y derecha-derecha; las rotaciones dobles resuelven los casos izquierda-derecha y derecha-izquierda. En el proyecto se decidió contabilizar rotaciones simples y dobles por separado, lo cual es útil para interpretar el costo adicional que el AVL paga a cambio de mantener una mejor altura. Este detalle fortalece el análisis, pues no basta con decir que el AVL es más eficiente; también debe explicarse por qué lo es y cuál es el trabajo extra que realiza para lograrlo.

La eliminación en AVL es conceptualmente más compleja que en BST porque, después de remover el nodo correspondiente, también es necesario revisar y restaurar el balance en los ancestros afectados. La implementación sigue la lógica del BST para localizar y sustituir nodos, y luego recalcula alturas y aplica rotaciones cuando se detecta desbalance. El uso del predecesor inorden para reemplazo es una variación válida con respecto al BST previo, siempre que se preserve la propiedad de orden.

Desde el punto de vista didáctico, el AVL fue la estructura idónea para demostrar la relación entre forma del árbol y costo temporal. Mientras el BST depende fuertemente del orden de inserción, el AVL conserva una altura acotada incluso bajo inserciones ordenadas. Esta estabilidad se evidencia tanto en la teoría de complejidad como en el benchmarking.

6.3 Árbol Multi-camino (N-ario)

Un árbol multi-camino es una estructura jerárquica donde cada nodo puede tener más de dos hijos. A diferencia de un BST o un AVL, su propósito principal no es optimizar búsqueda por orden, sino representar relaciones de pertenencia o contención. En el proyecto, esta estructura se eligió para modelar la jerarquía urbana de la ciudad: una ciudad contiene distritos, los distritos contienen zonas, las zonas contienen avenidas y las avenidas contienen intersecciones.

La implementación parte de un nodo N-ario que conoce tres elementos fundamentales: el dato que representa, el nivel jerárquico al que pertenece y la lista de hijos. Adicionalmente, cada nodo conserva un máximo de hijos permitido, lo que responde al requisito de número configurable de hijos. Aunque este máximo no representa una restricción real del urbanismo, sí resulta útil para demostrar que la estructura puede adaptarse a distintas configuraciones sin cambiar su lógica básica.

La clase ArbolN_ario<T> incluye un método para establecer la raíz, que en la demostración corresponde a la ciudad, y dos variantes de inserción. En una de ellas, el nivel del hijo se infiere a partir del nivel del padre; en la otra, el nivel del hijo se pasa explícitamente. El diseño de ambas opciones permite controlar mejor la construcción del árbol y defender que la estructura no se llena de nodos arbitrariamente, sino siguiendo reglas coherentes con la jerarquía urbana.

La búsqueda en un árbol N-ario no puede aprovechar una propiedad de orden como en los árboles binarios de búsqueda. Por ello, se implementa una búsqueda recursiva en profundidad que examina el nodo actual y luego recorre sus hijos. Esto implica que la búsqueda, en el peor caso, puede requerir visitar todos los nodos del árbol. Lejos de ser una debilidad del código, este comportamiento es una consecuencia natural de la estructura y de su propósito semántico.

Se implementaron recorridos pre-order, post-order y por niveles. El recorrido por niveles es especialmente relevante porque permite visualizar la ciudad capa por capa, algo útil tanto para el análisis estructural como para la presentación del sistema durante la defensa. Asimismo, se añadieron consultas específicas: conteo de intersecciones por distrito, profundidad máxima de la red, número de hojas, nodos internos y factor promedio de ramificación. Estas operaciones son importantes porque convierten al árbol N-ario en algo más que una representación visual; lo transforman en una estructura consultable y medible.

6.4 Cola de Prioridad basada en Heap

Una cola de prioridad es una estructura en la que la salida de los elementos no depende del orden de llegada, sino del criterio de prioridad. Este concepto se ajusta de manera directa al problema de tráfico urbano, donde una ambulancia o un accidente grave no deben esperar detrás de un reporte general solo porque fueron registrados después. La implementación se realizó sobre un heap binario almacenado en un ArrayList, lo cual permite representar el árbol implícitamente sin necesidad de nodos con punteros.

El heap puede entenderse como un árbol casi completo cuya raíz contiene siempre el elemento de mayor prioridad en el caso de un max-heap. La clase ColaPrioridad<T> utiliza un Comparator<T> para decidir qué significa “mayor prioridad”, haciendo posible intercambiar entre distintos criterios sin reescribir la estructura. Esta elección es especialmente valiosa en el proyecto porque el enunciado exige cambio dinámico de criterio.

La inserción consiste en agregar el elemento al final del arreglo interno y luego “subirlo” mediante intercambios mientras tenga mayor prioridad que su padre. Este proceso conserva la forma del árbol y restaura la propiedad de heap. Cada intercambio se contabiliza, lo que permitió registrar una métrica adicional más allá del tiempo total: el esfuerzo estructural requerido para acomodar los eventos.

La extracción del elemento de máxima prioridad se realiza tomando la raíz, moviendo el último elemento a la primera posición y luego “hundiendo” ese elemento hasta restaurar el heap. Esta operación es eficiente porque evita ordenar todo el conjunto y solo corrige el camino necesario. La diferencia frente a una lista ordenada es importante: el heap se diseñó precisamente para que inserciones y extracciones sean razonablemente rápidas en contextos de atención continua.

También se implementó modificación de prioridad. En esta solución, la cola localiza el elemento, asume que su prioridad ya fue ajustada externamente y entonces intenta reposicionarlo subiéndolo o bajándolo. Aunque la búsqueda del elemento es lineal, el reacomodo posterior conserva la lógica del heap. Además, la clase permite cambiar de max-heap a min-heap actualizando el comparador y reconstruyendo el heap mediante heapify de Floyd. Desde el punto de vista académico, esta funcionalidad demuestra dominio del tema y comprensión de que la prioridad no siempre será definida por un único criterio rígido.

6.5 Clases auxiliares y de soporte

Aunque el proyecto se concentra en cuatro estructuras principales, la solución incorporó clases de soporte que mejoran modularidad, reutilización y claridad. CentroComparaciones funciona como una fábrica de comparadores para Interseccion y Evento; su papel es centralizar políticas de ordenamiento y evitar que las estructuras dependan de decisiones de comparación codificadas internamente. Esto es especialmente pertinente cuando se desea reordenar por ID, congestión, riesgo, tiempo o prioridad.

GeneradorDatos se desarrolló para crear escenarios reproducibles de prueba. Utiliza una semilla fija en Random, lo que permite repetir experimentos con resultados consistentes. La clase produce intersecciones aleatorias, intersecciones ordenadas y descendentes, además de eventos aleatorios asociados a intersecciones. Esta automatización fue clave para poder ejecutar pruebas de 1,000; 10,000; 50,000 y 100,000 elementos sin construir datos manualmente.

Cronometro encapsula el uso de System.nanoTime() y genera objetos Medicion que registran etiqueta, tiempo total, cantidad de elementos y número de operaciones. Su valor académico radica en que transforma el benchmarking en un proceso sistemático: no solo se mide tiempo, también se calcula ns/op, se imprimen tablas y se exportan resultados a CSV. BenchMark, por su parte, coordina experimentos completos y ofrece métodos separados para BST, AVL, comparación BST vs AVL y cola de prioridad.

Por último, pero no menos importante, la clase Main integra todos los componentes mediante un menú de consola. Esto permitió demostrar el sistema en ejecución, cargar archivos, elegir entre BST y AVL, procesar eventos en tiempo real y ejecutar pruebas comparativas. Desde el punto de vista del documento técnico, el Main evidencia que las estructuras no se desarrollaron de manera aislada, sino integradas dentro de un flujo de uso comprensible.

7. Simulación y benchmarking

La parte experimental del proyecto se diseñó para validar que las estructuras implementadas no solo fueran correctas en teoría, sino también observables en ejecución. Para ello se desarrolló un módulo de benchmarking que automatiza generación de datos, inserciones, búsquedas, extracciones y exportación de resultados. El objetivo de esta sección no fue producir números decorativos, sino construir evidencia que permita interpretar por qué una estructura es más apropiada que otra en determinadas condiciones.

El sistema simula cuatro tamaños de carga para intersecciones: 1,000; 10,000; 50,000 y 100,000 elementos. Estos tamaños son suficientes para observar tendencias de crecimiento y diferencias entre estructuras sin quedarse únicamente en ejemplos pequeños. En el caso de eventos, se trabajó con 1,000; 5,000 y 10,000, con énfasis especial en el procesamiento de 10,000 eventos, tal como lo pide el enunciado.

Para el BST se midió inserción aleatoria, búsqueda sobre una muestra de elementos e inserción ordenada como representación del peor caso. Para el AVL se ejecutaron pruebas equivalentes, incluyendo conteo de rotaciones y comparación de altura frente al BST. Para la cola de prioridad se midió inserción en heap, extracción completa y comparación conceptual con una lista ordenada. Todas las mediciones se realizaron mediante la clase Cronometro, basada en System.nanoTime(), y los resultados se almacenan como objetos Medicion para su posterior exportación a CSV.

Desde la perspectiva metodológica, es importante subrayar que el benchmarking no sustituye al análisis teórico. Los tiempos medidos dependen del equipo, la carga del sistema operativo y el entorno de ejecución. Sin embargo, sí permiten observar patrones consistentes: por ejemplo, que el BST empeora dramáticamente bajo inserción ordenada, que el AVL mantiene una altura mucho menor en escenarios extremos y que el heap resuelve adecuadamente el problema de priorización de eventos sin ordenar una lista completa en cada operación.

La interpretación de los resultados debe hacerse con criterio. Si un BST y un AVL muestran tiempos cercanos en tamaños pequeños, ello no invalida la teoría; simplemente significa que el costo constante y el tamaño aún no son suficientes para separar fuertemente ambos comportamientos. A medida que crece la entrada, la altura y el número de comparaciones se vuelven más determinantes. Este tipo de reflexión es justamente el valor formativo del benchmarking dentro del curso.

Prueba	Qué se mide	Interpretación esperada
BST inserción aleatoria	Tiempo total, inserciones, altura resultante.	Aproxima el caso promedio; la altura suele ser razonable pero no garantizada.
BST inserción ordenada	Tiempo total, comparaciones, altura en peor caso.	El árbol tiende a degenerarse y la altura puede acercarse a n.
AVL inserción aleatoria	Tiempo, rotaciones, altura.	Mantiene balance con costo adicional moderado por rotaciones.
AVL inserción ordenada	Tiempo, altura y rotaciones.	Aun con entrada adversa la altura se conserva logarítmica.
Búsqueda BST vs AVL	Tiempo de búsqueda y comparaciones.	El AVL tiende a mostrar mayor estabilidad bajo grandes volúmenes.

Heap eventos	Tiempo de inserción/extracción e intercambios.	La prioridad se atiende eficientemente sin ordenamiento total repetido.

8. Análisis formal de complejidad

En esta sección se presenta el análisis formal de complejidad temporal y espacial de las estructuras principales del sistema. Para cada una se identifican mejor caso, caso promedio y peor caso, expresados mediante notación Ω, Θ y O. Se incluyen además explicaciones cualitativas para evitar que la tabla se convierta en un listado mecánico sin conexión con la implementación.

8.1 Complejidad del BST

Tabla 1. Complejidad temporal del BST

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Buscar	El elemento se encuentra en la raíz.	Ω(1)	Árbol razonablemente balanceado.	Θ(log n)	Árbol degenerado tipo lista.	O(n)
Insertar	Se inserta tras muy pocas comparaciones.	Ω(1)	Inserción en árbol de altura logarítmica.	Θ(log n)	Cadena lineal por inserción adversa.	O(n)
Eliminar	Nodo hoja o cercanísimo a la raíz.	Ω(1)	Localización y ajuste en altura logarítmica.	Θ(log n)	Localización en árbol degenerado.	O(n)
In-order / Pre / Post	Se visitan todos los nodos.	Ω(n)	Se visitan todos los nodos.	Θ(n)	Se visitan todos los nodos.	O(n)
Altura / conteo	Debe recorrerse toda la estructura usada por BFS.	Ω(n)	Recorrido completo.	Θ(n)	Recorrido completo.	O(n)

El factor crítico del BST es la altura. Si la altura h se mantiene baja, entonces buscar, insertar y eliminar cuestan proporcionalmente a h. Cuando la entrada llega en orden favorable, h suele comportarse como log n; cuando llega en orden adverso, h puede convertirse en n. Esto explica por qué el árbol es eficiente en teoría solo bajo ciertas distribuciones de entrada.

En la implementación desarrollada, la inserción se hizo iterativa para evitar desbordamiento de pila bajo entradas grandes. Este detalle no cambia la complejidad asintótica, pero sí mejora robustez práctica. Los recorridos y cálculos como altura o conteo de nodos visitan la estructura completa, por lo que su complejidad es lineal.

En cuanto a espacio, el BST requiere O(n) para almacenar n nodos. Adicionalmente, los recorridos iterativos pueden requerir O(h) u O(n) espacio auxiliar dependiendo de si emplean pila o cola y de la forma del árbol. En el peor caso de un recorrido por niveles, la cola puede crecer proporcionalmente al número de nodos del nivel más ancho.

8.2 Complejidad del AVL

Tabla 2. Complejidad temporal del AVL

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Buscar	Puede hallarse cerca de la raíz.	Ω(1)	Árbol balanceado por definición.	Θ(log n)	Aun en peor caso mantiene altura logarítmica.	O(log n)
Insertar	Pocas comparaciones y sin rotación significativa.	Ω(log n)	Localización más rebalanceo acotado.	Θ(log n)	Búsqueda de posición y rotaciones locales.	O(log n)
Eliminar	Ajuste mínimo tras remover nodo favorable.	Ω(log n)	Localización y rebalanceo.	Θ(log n)	Revisión de ancestros y posibles rotaciones.	O(log n)
Rotar	Operación local sobre pocos punteros.	Ω(1)	Operación local.	Θ(1)	Operación local.	O(1)
Recorridos / altura almacenada	Visita total de nodos en recorridos.	Ω(n)	Recorridos completos.	Θ(n)	Recorridos completos.	O(n)

La ventaja fundamental del AVL es que garantiza una altura O(log n). Por ello, las operaciones principales no se degradan a lineales aun cuando la entrada llegue ordenada. Este comportamiento se logra a costa de mantener una altura en cada nodo y ejecutar rotaciones cuando el factor de balance sale del rango permitido.

Las rotaciones, consideradas individualmente, son operaciones O(1) porque reacomodan un número constante de referencias. Sin embargo, la inserción o la eliminación completas siguen costando O(log n), debido a que primero debe localizarse la posición correcta dentro del árbol. Lo importante es que el rebalanceo no cambia el orden de magnitud de la operación.

En términos de espacio, el AVL sigue siendo O(n) porque almacena un nodo por elemento. Su costo adicional respecto del BST consiste en un campo extra de altura por nodo. Este incremento es constante por elemento y, por lo tanto, no modifica la complejidad espacial asintótica.

8.3 Complejidad del árbol N-ario

Tabla 3. Complejidad temporal del árbol N-ario

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Buscar nodo	La raíz coincide con el dato.	Ω(1)	Se recorre una porción significativa del árbol.	Θ(n)	El dato está al final o no existe.	O(n)
Insertar hijo	Padre conocido de inmediato y con espacio.	Ω(1)	Debe buscarse el padre antes de insertar.	Θ(n)	Padre ausente o muy profundo.	O(n)
Pre-order / Post-order	Visita completa por diseño.	Ω(n)	Visita completa.	Θ(n)	Visita completa.	O(n)
Por niveles	Visita completa mediante cola.	Ω(n)	Visita completa.	Θ(n)	Visita completa.	O(n)
Profundidad / conteos	Recorrido total de nodos relevantes.	Ω(n)	Recorrido total.	Θ(n)	Recorrido total.	O(n)

A diferencia de BST y AVL, el árbol N-ario no tiene un criterio de orden que permita descartar subárboles enteros a partir de una comparación. Por esta razón, sus búsquedas y varias de sus consultas son lineales en el número total de nodos en el peor caso. Esta complejidad no indica una mala implementación, sino una consecuencia del tipo de estructura y de su semántica jerárquica.

La inserción puede parecer una operación constante cuando el padre ya se conoce, ya que agregar un hijo al final de una lista es O(1). Sin embargo, en la interfaz pública del proyecto normalmente primero debe localizarse el padre dentro del árbol, por lo que el costo real de la operación completa se vuelve lineal en el peor caso.

En cuanto a espacio, la estructura requiere O(n) para n nodos, además del espacio de las listas de hijos. Si se considera la suma total de referencias almacenadas en dichas listas, sigue siendo lineal respecto al número de conexiones reales entre nodos.

8.4 Complejidad de la cola de prioridad (heap)

Tabla 4. Complejidad temporal de la cola de prioridad

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Peek	Consulta directa de la raíz.	Ω(1)	Consulta directa.	Θ(1)	Consulta directa.	O(1)
Insertar	No necesita subir o sube poco.	Ω(1)	Subida por altura del heap.	Θ(log n)	Sube hasta la raíz.	O(log n)
Extraer máximo	Reacomodo mínimo tras mover el último.	Ω(1)	Hundimiento por altura del heap.	Θ(log n)	Baja hasta hoja profunda.	O(log n)
Modificar prioridad	Elemento cercano y cambio mínimo.	Ω(1)	Búsqueda lineal más ajuste.	Θ(n)	Localización completa más reacomodo.	O(n)
Cambiar comparador + reconstruir	Heapify de estructura existente.	Ω(n)	Reconstrucción completa.	Θ(n)	Reconstrucción completa.	O(n)

El heap mantiene equilibrio estructural por construcción, ya que siempre representa un árbol casi completo. Por esta razón, la altura es logarítmica y tanto insertar como extraer recorren a lo sumo esa altura. Esta propiedad es precisamente la que vuelve útil al heap para sistemas con inserciones y extracciones frecuentes.

La operación modificarPrioridad no conserva costo logarítmico en la implementación desarrollada porque primero necesita localizar el elemento dentro del arreglo interno. Dicha búsqueda es lineal. Una vez encontrado, el reacomodo sí es logarítmico. Por ello, el costo total queda dominado por la fase de búsqueda.

El cambio dinámico de comparador se resolvió reconstruyendo el heap con heapify de Floyd, operación lineal. Esta decisión es sólida desde el punto de vista algorítmico porque evita volver a insertar uno por uno todos los elementos.

8.5 Complejidad de clases auxiliares relevantes

Tabla 5. Complejidad resumida de utilidades auxiliares

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Generar n intersecciones	Construcción de lista completa.	Ω(n)	Generación con operaciones constantes por elemento.	Θ(n)	Generación completa.	O(n)
Mezclar lista	Debe visitar elementos para permutar.	Ω(n)	Collections.shuffle.	Θ(n)	Collections.shuffle.	O(n)
Exportar CSV	Escritura por medición.	Ω(m)	m filas de resultados.	Θ(m)	m filas de resultados.	O(m)
Imprimir tabla de mediciones	Recorre todas las mediciones.	Ω(m)	Recorrido completo.	Θ(m)	Recorrido completo.	O(m)

Aunque estas clases no sean estructuras principales, su análisis demuestra comprensión global del sistema. La generación de datos y la exportación de resultados son lineales respecto al tamaño del conjunto que procesan, lo cual es coherente con su función de apoyo.

9. Buenas prácticas y decisiones de ingeniería aplicadas

Antes de presentar las conclusiones, es importante destacar las decisiones de calidad que se incorporaron en el proyecto, pues la robustez de una solución no depende únicamente de que las estructuras sean correctas desde el punto de vista teórico. También importa la manera en que el código se organiza, se protege frente a errores y se prepara para la demostración y el análisis.

En primer lugar, se aplicó separación de responsabilidades. Las estructuras de datos no dependen de la clase Main ni de la forma en que se imprime un menú. Las entidades de dominio tampoco contienen lógica de benchmarking. Esta división favorece la comprensión del código y hace que cada componente tenga un propósito claro. Para un proyecto grupal, esta organización también facilita la colaboración y la defensa conjunta, ya que cada integrante puede explicar un bloque bien delimitado.

En segundo lugar, se utilizaron Generics y Comparator de manera consistente. Esto evitó duplicar implementaciones por tipo de dato y reforzó el uso de inyección de comportamiento. En vez de modificar internamente un BST para ordenar por distintas variables, se decidió reutilizar la misma estructura con comparadores diferentes, centralizados en CentroComparaciones.

Se incorporaron métricas internas dentro de las estructuras, como comparaciones, rotaciones, inserciones, extracciones e intercambios. Esto no es obligatorio para que una estructura funcione, pero sí es una buena práctica cuando se pretende analizar rendimiento con fundamento. Gracias a estas métricas, el informe puede discutir más que tiempos de reloj; puede hablar de trabajo algorítmico observado.

También se tomaron medidas de robustez. Por ejemplo, la inserción iterativa en el BST reduce el riesgo de problemas de pila en cargas grandes; el uso de try-with-resources en la carga de archivos asegura el cierre automático de recursos; y varios setters validan rangos válidos para evitar estados inconsistentes. De igual forma, la generación de datos pseudoaleatorios con semilla fija fortalece la reproducibilidad de las pruebas.

Desde la perspectiva de mantenibilidad, el proyecto incluyó métodos de impresión de estadísticas, toString descriptivos y módulos de demostración dentro del Main. Estas decisiones son útiles durante la defensa, pero también durante el desarrollo, pues facilitan depuración, validación visual y detección de comportamientos inesperados.

Buena práctica	Manifestación concreta en el proyecto
Separación de responsabilidades	Estructuras, entidades, benchmarking, comparadores y menú principal en clases distintas.
Programación genérica	Uso de <T> en BST, AVL, Nodo, NodoN_ario y ColaPrioridad.
Delegación de criterio	Comparator<T> y clase CentroComparaciones para no acoplar la estructura a un solo orden.
Validación de entradas	Límites en prioridad, riesgo, velocidad y volumen; manejo de NumberFormatException en carga de archivos.
Reproducibilidad	Random con semilla fija para repetir escenarios de prueba.
Medición formal	Cronometro con ns/op, tablas y exportación CSV.
Interfaz demostrativa	Main modular con menús por estructura, benchmarking e integración.

10. Manual de funcionalidad del sistema

En esta sección se describe el uso del programa desde la perspectiva del usuario. El propósito no es únicamente indicar qué botón u opción existe, sino explicar qué debe ingresar la persona que ejecuta la aplicación, qué resultado se espera en pantalla y qué debe interpretarse de cada salida. El sistema fue diseñado como una aplicación de consola, por lo que la interacción se basa en menús y entrada numérica.

10.1 Inicio del sistema

Al ejecutar la aplicación se muestra un banner institucional con el nombre del sistema MIGTU y, a continuación, el menú principal. El usuario debe seleccionar una opción numérica y presionar Enter. Si se ingresa un valor no numérico o una opción no contemplada, el programa responde con un mensaje de opción no válida y vuelve a mostrar el menú.

El menú principal contiene seis módulos: BST, AVL, árbol multi-camino, cola de prioridad, simulación y benchmarking, e integración final del sistema. La opción 0 permite terminar la ejecución. La interfaz fue diseñada de esta forma para que cada estructura pueda demostrarse por separado antes de pasar a la integración completa.

10.2 Módulo BST

1. Demo inserción aleatoria (1,000).

Al seleccionar esta opción, el sistema genera 1,000 intersecciones pseudoaleatorias y las inserta en un BST ordenado inicialmente por ID. Luego cronometra la inserción, cambia el comparador a nivel de congestión ascendente, limpia el árbol y vuelve a insertar los mismos datos mezclados. El usuario no necesita escribir nada adicional. En pantalla se observa un resumen temporal y estadísticas del árbol. Lo que debe entenderse es que el BST puede reutilizarse con otro criterio, pero para que el nuevo orden sea coherente el árbol debe reconstruirse.

2. Demo inserción ordenada (peor caso).

Esta opción genera 1,000 intersecciones ordenadas por ID y las inserta de forma secuencial. El programa muestra el tiempo medido e imprime una advertencia de que, bajo inserción ordenada, el BST se degenera hacia una forma lineal. El usuario debe observar especialmente la altura del árbol y el número de comparaciones, ya que estos indicadores explican el peor caso teórico.

3. Mostrar recorridos.

Se crean 20 intersecciones aleatorias y luego se imprimen los recorridos in-order, pre-order, post-order y por niveles. El usuario debe interpretar que el in-order devuelve los elementos ordenados según el comparador activo, mientras que los otros recorridos muestran distintas estrategias de visita del árbol. Esta opción es útil para validar visualmente la estructura interna.

4. Mostrar estadísticas.

El sistema carga 10,000 intersecciones aleatorias y despliega altura, nodos, hojas, balance, comparaciones, inserciones y búsquedas. La finalidad es ofrecer una vista resumida del estado del BST bajo una carga significativa.

10.3 Módulo AVL

5. Demo inserción aleatoria (1,000).

Genera 1,000 intersecciones aleatorias, las inserta en un AVL y muestra el tiempo total. Después imprime estadísticas generales. El usuario debe comprender que, aun cuando el árbol reciba los datos en orden cualquiera, la estructura mantiene balance gracias a su mecanismo interno de rotaciones.

6. Demo inserción ordenada (AVL balancea).

Esta opción evidencia la diferencia con el BST. Se insertan intersecciones en orden ascendente, pero la altura del árbol se mantiene controlada. El sistema muestra tiempo, nota explicativa y estadísticas específicas del AVL, incluyendo rotaciones simples, dobles y factor de balance de la raíz.

7. Comparación altura BST vs AVL.

Se construyen simultáneamente un BST y un AVL con la misma secuencia ordenada de 1,000 nodos. Luego se imprime una tabla comparativa con altura, número de nodos, rotaciones del AVL, balance y comparaciones. El usuario debe fijarse en que ambos reciben exactamente la misma entrada, pero responden de forma estructural muy distinta.

8. Mostrar estadísticas AVL.

Carga 10,000 intersecciones aleatorias en un AVL y presenta un resumen del estado estructural. Es una opción útil cuando se necesita evidenciar estabilidad bajo una cantidad mayor de datos.

10.4 Módulo árbol multi-camino

9. Demo jerarquía urbana.

Se crea una ciudad de demostración llamada Guatemala y se insertan distritos, zonas, avenidas e intersecciones. La salida se presenta con sangría, de manera que el usuario visualiza la jerarquía como si fuera un árbol textual. No se solicita entrada adicional. Esta opción sirve para evidenciar que el árbol N-ario modela pertenencia y no ordenamiento.

10. Recorrido por niveles.

Se imprime la ciudad agrupada por niveles. La salida aparece como una lista de listas: primero la ciudad, luego los distritos, después las zonas, avenidas e intersecciones. El usuario debe interpretar cada fila como una capa jerárquica del árbol.

11. Recorrido en profundidad.

El sistema ejecuta un recorrido DFS pre-order e imprime el camino de visita. Al final muestra la profundidad máxima de la red. Esto permite al usuario comprender la diferencia entre explorar en anchura y en profundidad.

12. Consultas específicas.

Se muestran resultados como número de intersecciones en Distrito Norte, número de intersecciones en Distrito Sur, profundidad máxima y total de intersecciones. Estas consultas evidencian que la estructura no solo se usa para mostrar la ciudad, sino también para responder preguntas concretas.

13. Estadísticas del árbol.

Imprime cantidad máxima de hijos, profundidad máxima, número de nodos, hojas, nodos internos y factor promedio de ramificación. El usuario debe entender que estas métricas describen la forma de la red urbana modelada.

10.5 Módulo cola de prioridad

14. Demo inserción y extracción.

Se generan 10 eventos aleatorios y se insertan en una cola de prioridad configurada como max-heap. Durante la inserción se imprime el tipo de evento y su prioridad. Luego se extraen uno por uno mostrando que salen en orden de prioridad. El usuario debe observar que el orden de salida no coincide necesariamente con el orden de llegada.

15. Demo modificar prioridad.

Se insertan cinco eventos, se consulta el próximo a salir y posteriormente se toma uno de los eventos ya existentes, se aumenta su prioridad a 999 y se invoca la actualización de posición en el heap. La salida muestra el antes y el después. Esta opción enseña que la cola puede reacomodarse cuando cambia la urgencia de un evento.

16. Demo cambio dinámico de criterio.

Inicialmente la cola opera como max-heap. Luego el comparador se cambia para comportarse como min-heap y se muestra el nuevo próximo a salir. Después se restablece el criterio original. El usuario debe comprender que la prioridad depende del comparador activo, no de una lógica fija escrita dentro del heap.

17. Procesar 10,000 eventos.

Se generan 10,000 eventos, se mide el tiempo de inserción y el tiempo de extracción total. La salida incluye eventos procesados e intercambios realizados. Esta opción sirve como evidencia de correcto funcionamiento y como demostración de eficiencia en alta carga.

18. Estadísticas de la cola.

Se cargan 1,000 eventos y se imprime el tamaño actual, cantidad de inserciones, extracciones, intercambios totales y el próximo evento a salir. Es una vista resumida del estado del heap.

10.6 Módulo de simulación y benchmarking

Este módulo agrupa todas las pruebas experimentales. La opción de benchmark completo ejecuta todas las mediciones del proyecto y exporta un archivo CSV con los resultados. Las demás opciones permiten correr solo BST, solo AVL, comparación BST vs AVL o únicamente la cola de prioridad. El usuario no debe proporcionar datos manuales; el sistema genera automáticamente los escenarios.

La salida típica consiste en tablas de mediciones con etiquetas, tiempo en milisegundos, cantidad de elementos, número de operaciones y nanosegundos por operación. Lo importante para el usuario es interpretar tendencias y no memorizar un número exacto, dado que la variación del entorno puede cambiar los tiempos absolutos.

10.7 Módulo de integración final del sistema

19. Cargar ciudad desde archivo.

El sistema solicita la ruta de un archivo .csv o .txt. El formato esperado es id,nombre,avenida,zona,distrito, con encabezado en la primera línea. Si la ruta no existe, se informa al usuario. Si el archivo contiene líneas vacías o columnas insuficientes, dichas líneas cuentan como errores y se omiten. Al finalizar, el programa indica cuántas intersecciones se cargaron, cuántos errores hubo y qué estructura de indexación se utilizó.

20. Elegir estructura (BST o AVL).

Se muestra un pequeño submenú con dos opciones: BST y AVL. Según la selección, el índice principal del sistema se crea como un BST o como un AVL ordenado por ID. Esta decisión afecta la estructura sobre la cual se cargarán posteriormente las intersecciones desde archivo.

21. Procesar eventos en tiempo real.

Se generan 1,000 eventos aleatorios, se insertan en la cola de prioridad y se mide el tiempo total. Después se construye una copia del heap para extraer e imprimir los cinco eventos de mayor prioridad sin destruir la cola original. El usuario verá en pantalla el tipo de evento, su prioridad y el ID de la intersección asociada. Esta opción resume de forma compacta la lógica operacional del sistema.

22. Mostrar estadísticas estructurales.

Si existe una estructura de indexación activa, el sistema imprime sus estadísticas. Si la estructura activa es un AVL, se añaden sus métricas específicas de rotaciones y factor de balance. Si ya se construyó la ciudad o la cola de eventos, también se muestran sus respectivas estadísticas. El usuario debe usar esta opción como resumen del estado global del sistema.

10.8 Qué debe comprender el usuario al utilizar el programa

- Que BST y AVL almacenan intersecciones, pero responden distinto cuando la entrada es adversa.
- Que el árbol N-ario no ordena datos, sino que representa la jerarquía urbana.
- Que la cola de prioridad atiende urgencia y no orden de llegada.
- Que las estadísticas presentadas no son decorativas: reflejan altura, comparaciones, rotaciones e intercambios.
- Que el benchmarking produce evidencia experimental que debe interpretarse junto con la teoría de complejidad.
- Que el archivo cargado desde integración debe respetar el formato de columnas esperado para evitar errores de lectura.

11. Conclusiones generales

Se concluye que el proyecto permitió demostrar con claridad que no existe una única estructura de datos óptima para todos los problemas del sistema. Cada necesidad del dominio urbano exigió una estructura distinta: el BST y el AVL para indexación, el árbol N-ario para modelado jerárquico y el heap para priorización de eventos. Esta correspondencia entre problema y estructura fue uno de los aprendizajes centrales del trabajo.

El BST evidenció ser una estructura conceptualmente simple y eficiente cuando la altura se mantiene baja, pero también mostró su fragilidad ante inserciones ordenadas. Esta limitación justificó plenamente la implementación del AVL, cuya principal fortaleza fue mantener estabilidad estructural incluso bajo cargas adversas. La comparación experimental entre ambos árboles permitió observar de forma concreta el valor del balance automático.

El árbol N-ario demostró ser la opción adecuada para representar relaciones de pertenencia dentro de la ciudad. Aunque sus búsquedas no pueden competir en complejidad con las de un BST o un AVL, su propósito no es ese. Su verdadero valor radica en expresar la jerarquía de forma natural y permitir consultas semánticas sobre distritos, zonas, avenidas e intersecciones.

La cola de prioridad confirmó que los eventos críticos no deben administrarse como una cola FIFO tradicional. El uso de un heap permitió insertar y extraer eventos con eficiencia, mantener una política de urgencia y demostrar cambio dinámico de criterio. Esto resulta especialmente coherente con el contexto del problema, donde la prioridad puede variar según el tipo de incidente o el tiempo de reporte.

Desde el punto de vista metodológico, el benchmarking fue una parte indispensable. Más que producir cifras, permitió articular la teoría de complejidad con observaciones concretas. El uso de comparadores, clases auxiliares, generación de datos reproducible y una interfaz de integración fortaleció la calidad del proyecto y evidenció una solución razonada y también, defendible.